

IGNOU SYNOPSIS

STUDENT NAME: SHAIKH IRAM JALEEL AHMED

ENROLLMENT NUMBER: 2450540111

PROGRAM CODE: MCA_NEW

COURSE CODE: MCSP-232

STUDY CENTRE CODE: 49053

.....

COURIER MANAGEMENT SYSTEM

MCSP -232

MCA_NEW

INDIRA GANDHI NATIONAL OPEN UNIVERSITY

NEW DELHI



Submitted By :

Name : SHAIKH IRAM JALEEL AHMED

Enrollment number : 2450540111

Under Guidance Of :

Name : VISHWJA SURALKAR

Graduation: B. E in Computer Engineering



SCHOOL OF COMPUTER AND INFORMATION SCIENCES
IGNOU, MAIDAN GARHI, NEW DELHI - 110 068

II. PROFORMA FOR THE APPROVAL OF MCA PROJECT PROPOSAL (MCSP-232)

(Note: All entries of the proforma of approval should be filled up with appropriate and complete information.
Incomplete proforma of approval in any respect will be summarily rejected.)

Project Proposal No :
(for office use only)

Enrolment No.: 245054011
Study Centre: 49052
Regional Centre: 49: Mumbai
RC Code: 49: Mumbai
E-mail: Anamshah212@gmail.com
Mobile No: 8828752318

1. Name and Address of the Student: SHAIKH IRAM JALEEL AHMED, Bhashiya
Kamla Nagar, SP Road, Antop hill, Wadala (E), Mumbai - 400037.
2. Title of the Project***: COURIER MANAGEMENT SYSTEM
3. Name and Address of the Guide: NISHWJA SURAIKAR, #304-C, 3rd floor,
C-WING, Gokul Heaven, Thakur Complex, Kandivali (E) - Mum
400101
4. Educational Qualification of the Guide:
(Attach bio-data also)
(*in Computer Science / IT only)
- | | | | | |
|--------------------------|--------------------------|-------------------------------------|--------------------------|--------------------------|
| Ph.D.* | M.Tech.* | B.E*/B.Tech.* | MCA | M.Sc.* |
| ☐ | ☐ | ☒ | ☐ | ☐ |
5. Working / Teaching experience of the Guide**: 5+ years
6.
7. (** Note: At any given point of time, a guide should not provide guidance for more than 5 MCA students of IGNOU)
8. Software used in the Project*** : Django, MySQL, Python, HTML, CSS, JAVASCRIPT
(*** Please refer to section VIII of these guidelines)

12. Is this your first submission?



10. Yes



No

Signature of the Student
Date: 19/12/2025

Signature of the Guide
Date: 19/12/2025

For Office Use Only

Name:



Approved

Not Approved

Signature, Designation, Stamp of the
Project Proposal Evaluator

Date:

Suggestions for reformulating the Project:

VISHWJA SURALKAR

Email: vishwja.vss@gmail.com

LinkedIn Profile: www.linkedin.com/in/vishwjasuralkar

Objective:

To be a part of a growth oriented and progressive company or organization to enhance my working skills and aspire for more senior roles with utmost sincerity and dedicated smart work in future.

Work Experience: Total: 5 Years +

- Working as a Programming Assistant (Front-end & Back-end) at the **Institute of Banking Personnel Selection (IBPS), Mumbai** at Grade B since 20th Mar 2023.
Nature of Work: Working in different departments to design and develop Website/Online database tools based on Python-Django Web Framework, Python3, MySQL, HTML5, JQuery, JavaScript, CSS3, and Bootstrap 4. Writing Python-based program scripts to solve internal department related tasks.
 - Worked as an Associate Software Engineer (ASE) Trainee in **Tata Consultancy Services (TCS), Pune** from 9th Jan 2020 to 4th Jan 2022.
Nature of Work: Worked on software development for different projects as per the requirement of the company based on Java, HTML5, JQuery, JavaScript, CSS3.
 - Worked as an Engineer Trainee at **Cognizant, Pune** from 9th May 2019 to 31st Dec 2019.
Nature of Work: Worked in Batch-Operations for Job scheduling, running and monitoring of Batch jobs.
-

Skills:

- **Languages:**
 - **Front-end:** HTML5, JSP and CSS3, Django template.
 - **Scripting:** JavaScript, JQuery.
 - **Programming:** Python3, C, C++, Java, SQL.
 - **Framework:** Python-Django Web Framework, Bootstrap 4.
 - **Databases:** MySQL.
 - **Tools:** Microsoft Visual Studio, Oracle SQL Developer, Eclipse IDE, Visual Studio Code, Git Extensions.
-

V Suralkar
19/12/25

Academic Information:

- Completed B.E. in Computer Engineering from MKSSS's Cummins College of Engineering for Women, Pune affiliated to The Savitribai Phule Pune University (formerly University of Pune) in 2018.

Achievements/Activities:

Co-founder @ www.travelouge.in

- We at www.travelouge.in provide travel blogs on different destination across the world.

College Level:

- Google India Challenge Scholarship 2018:
 - Google India has collaborated with online education Udacity to help train developers and students across India and create technically advanced workforce in India.
- Winner of Innovative Ideas for India's Swachh Bharat Abhiyan 2015-16:
 - This competition was organized by Dr. Raghunath Mashelkar. In this competition candidate should give innovative ideas which will help for Swachh Bharat Abhiyan.
- Organizer @ Code Club, CCOEW, Pune
 - Code Club conducts coding sessions and organizes IT workshop which will help students to enhance programming skills.
- Organizer @ Online Hackathon in Innovation 2k17 (College Technical fest)
 - Designed critical coding problems with test cases.
- Speaker @ Astronomy and Star gazing sessions
 - Conducted many star gazing sessions and Astronomy sessions more to create awareness among students.

Projects Undertaken:

- Depression Level using Social Media Posts (BE Project):
 - A self-diagnosis Web Application under Machine Learning domain which predict depression level of user by extracting Facebook data (maintaining all security concern) and analyzing questionnaires.
 - Classification is done using machine learning algorithm. Even with the report doctor's location nearby the user is mentioned.
- Travel Blog Website: www.travelogue.in
 - A website application which shows blogs on different destination across world.
- Website for College's Technical Fest Innovation2k17
 - Multipage website including welcome page, about and details regarding different events being held.
 - Integrated Google map feature for directions to college (Place of event).
- Sunshine App (Android)
 - Provide weekly forecast details in both temperature units (Celsius and Imperial) as per the option selected.
 - Select on date and get temperature, humidity, pressure, wind details.

Suralkar
19/12/25

Strengths:

- A sincere, hardworking and talented graduate Engineer with good academic background.
 - Good organizational skills and having logical approach for problem solving.
 - Ability to think through a problem coupled with confidence to make ideas heard.
 - Keen to enquire about new software and new technologies being developed.
-

Hobbies:

- Photography, Internet Surfing, Volunteer Work/Community Involvement & Listening Music.
- Indoor Activities - Sketching, Painting etc.
- Outdoor Activities - Star gazing, Jogging etc.

Personal Details:

Father's Name:	Shekhar Rambhau Suralkar
Mother's Name:	Jayashree
D.O.B:	19 th May 1996
Nationality:	Indian
Languages Known:	Marathi, Hindi & English
Communication Address:	#304-C, 3 rd Floor, C-Wing, Gokul Heaven, Thakur Complex Kandivali East, Mumbai, 400101
Permanent Address:	D/O Shekhar Suralkar, Plot No-25, Parakh Nagar, Jalgaon, Maharashtra, 425001

V. Suralkar
19/12/25

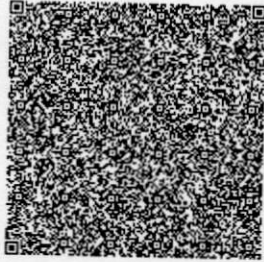


भारत सरकार
Government of India

भारतीय विशिष्ट ओळख प्राधिकरण
Unique Identification Authority of India

नोंदणी क्रमांक: / Enrolment No.: 2722/25001/00656

To
विश्वजा शेखर सुरळकर
Vishwja Shekhar Suralkar
plot n 25 sarve n 479/a,
parakh nagar,
VTC: Jalgaon,
PO: Jalgaon,
District: Jalgaon,
State: Maharashtra,
PIN Code: 425001,
Mobile: 9421513244



आपला आधार क्रमांक / Your Aadhaar No. :

2926 4110 3942

VID : 9165 4215 6269 0012

माझे आधार, माझी ओळख



भारत सरकार
Government of India



विश्वजा शेखर सुरळकर
Vishwja Shekhar Suralkar
जन्म तारीख/DOB: 19/05/1996
महिला/ FEMALE

आधार हा ओळखीचा पुरावा आहे, नागरिकत्व किंवा जन्मतारखेचा नाही.
हे फक्त पडताळणीसाठी वापरले जावे (ऑनलाइन प्रमाणीकरण किंवा QR कोडचे स्कॅनिंग/ ऑफलाइन XML)
Aadhaar is proof of identity, not of citizenship or date of birth. It should be used with verification (online authentication, or scanning of QR code / offline XML).

2926 4110 3942

माझे आधार, माझी ओळख



Government of India



माहिती / INFORMATION

- आधार हा ओळखीचा पुरावा आहे, नागरिकत्वाचा किंवा जन्मतारखेचा नाही. जन्मतारीख आधार क्रमांक धारकाने प्रस्तुत केलेल्या नियमांमध्ये निर्दिष्ट केलेल्या जन्मतारीख दस्तऐवजाच्या पुराव्याद्वारे समर्थित असलेल्या माहितीवर आधारित आहे.
- ह्या आधार पत्राची पडताळणी UIDAI-नियुक्त प्रमाणीकरण एजन्सीद्वारे ऑनलाइन प्रमाणीकरणाद्वारे किंवा ऑप स्टोअरमध्ये उपलब्ध mAadhaar किंवा Aadhaar QR स्कॅनर ऑप वापरून किंवा www.uidai.gov.in वर उपलब्ध सुरक्षित QR कोड रीडर ऑप वापरून QR कोड स्कॅनिंगद्वारे सत्यापित केले जावे.
- आधार अद्वितीय आणि सुरक्षित आहे.
- ओळख आणि पत्त्याला आधार देणारी कागदपत्रे आधार नोंदणीच्या तारखेपासून दर 10 वर्षांनी आधारमध्ये अद्यतनित केली जावीत.
- आधार तुम्हाला विविध सरकारी आणि गैर-सरकारी लाभ/सेवांचा लाभ घेण्यास मदत करते.
- आधारमध्ये तुमचा मोबाईल नंबर आणि ईमेल आयडी अपडेट ठेवा.
- आधार सेवांचा लाभ घेण्यासाठी mAadhaar ऑप डाउनलोड करा.
- आधार/बायोमेट्रिक्स वापरत नसताना सुरक्षितता सुनिश्चित करण्यासाठी लॉक/अनलॉक आधार/बायोमेट्रिक्सचे वैशिष्ट्य वापरा.
- आधारची मागणी करणाऱ्या संस्थांनी संमती घेणे बंधनकारक आहे.
- Aadhaar is proof of identity, not of citizenship or date of birth (DOB). DOB is based on information supported by proof of DOB document specified in regulations, submitted by Aadhaar number holder.
- This Aadhaar letter should be verified through either online authentication by UIDAI-appointed authentication agency or QR code scanning using mAadhaar or Aadhaar QR Scanner app available in app stores or using secure QR code reader app available on www.uidai.gov.in.
- Aadhaar is unique and secure.
- Documents to support identity and address should be updated in Aadhaar after every 10 years from date of enrolment for Aadhaar.
- Aadhaar helps you avail of various Government and Non-Government benefits/services.
- Keep your mobile number and email id updated in Aadhaar.
- Download mAadhaar app to avail of Aadhaar services.
- Use the feature of Lock/Unlock Aadhaar/biometrics to ensure security when not using Aadhaar/biometrics.
- Entities seeking Aadhaar are obligated to seek consent.

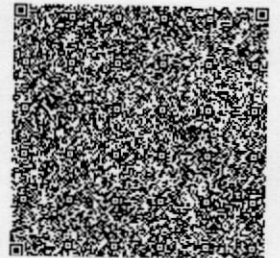


भारतीय विशिष्ट ओळख प्राधिकरण
Unique Identification Authority of India



पत्ता:
प्लॉट न २५ सर्वे न ४७९/अ, पारख नागर, जळगाव,
जळगाव, जळगाव,
महाराष्ट्र - 425001

Address:
plot n 25 sarve n 479/a, parakh nagar,
Jalgaon, PO: Jalgaon, DIST: Jalgaon,
Maharashtra - 425001



2926 4110 3942

VID : 9165 4215 6269 0012

1947

help@uidai.gov.in

www.uidai.gov.in

V. Suralkar
19/12/25

Contents

Abstract	2
Introduction	2
Literature Review	3
Problem Statement	3
Objectives.....	4
Scope and Limitations.....	5
Methodology	6
System Design.....	7
Context Diagram	7
Data Flow Diagrams.....	7
Use Case Diagram	10
ER (Entity-Relationship) Diagram.....	11
Class Diagram	12
Sequence Diagram.....	14
Activity Diagram	15
Component Diagram	16
System Implementation	17
Testing.....	18
Results and Discussion.....	19
Future Enhancements	20
Conclusion.....	21
References	21
Appendices	22

Courier Management System

Abstract

The Courier Management System (CMS) developed for this study is a Django-based web application designed to automate and optimize parcel delivery operations for small businesses and logistics providers. The study follows an Agile development lifecycle with iterative planning, requirement analysis, and testing phases. Key objectives included automating shipment booking, tracking, delivery assignment, and notifications to replace error-prone manual processes. The system integrates user roles (Admin, Manager, Customer, Courier) and features real-time tracking, invoicing, and Discord webhook notifications for status updates. In development, a combination of Django (Python), a relational database (MySQL/SQLite via Django ORM), and frontend technologies (HTML, CSS, JavaScript) was used. System design artifacts (UML diagrams, ERD, DFDs) guided implementation. The outcomes demonstrate that the CMS successfully streamlines end-to-end courier operations. Tasks such as booking a shipment, assigning a delivery agent, and generating invoices are now handled digitally with validation and security. Testing (unit, manual, and integration) verified functionality under various scenarios, with test cases confirming requirements coverage. Results show the system meets its goals: it significantly reduces manual effort, errors, and delays. As reported in related work, the system's automation **"saves time, effort, and cost"** and is **"user-friendly, secure, and efficient"**. In summary, the CMS achieves its objectives of improved efficiency, transparency, and reliability. It provides a foundation for future extensions (e.g. mobile apps, route optimization) to further advance courier logistics and customer experience.

Introduction

Efficient courier and parcel delivery services are critical in the current e-commerce and logistics landscape. In India, the Courier, Express, and Parcel (CEP) market is rapidly expanding due to rising online retail and improved infrastructure. Despite this growth, many small courier operators still rely on manual record-keeping and communication. Managing customer details, tracking shipments, coordinating delivery personnel, and handling billing by paper or basic spreadsheets can become **"quite complex,"** leading to **"significant inefficiencies, delays, increased costs, errors, [and] poor customer experience"**. For example, print-based logs and phone calls for status updates cause frequent delays and inaccuracies when staff are unavailable. Such drawbacks highlight the need for computerized systems. As

Ramya et al. observe, “**there is a pressing need for a detailed software way out that has the potential to automate and simplify courier business operations to overcome these challenges**”. This project is motivated by the industry demand for digital transformation: by automating routine tasks (order entry, tracking, assignments, invoicing), the system aims to eliminate manual errors, speed up processing, and improve transparency. The CMS thus addresses the problem context of outdated courier workflows, leveraging modern web technologies (Django/Python) to meet growing logistic demands and enhance the user experience.

Literature Review

Recent studies and projects demonstrate the benefits of automating courier services. Ramya *et al.* developed “**Courier Connect**” – a Django-based courier management platform – with modules for customer management, order tracking, courier personnel assignment, invoicing, and reporting. They report that Courier Connect “**streamlines the entire delivery process from order booking to final delivery**”, delivering insights that “**increase the efficiency and quality of the courier service**”. Similarly, Devadiga *et al.* presented an online Django system where users can send and track couriers in real time; their results showed an “**efficient and user-friendly solution**” that reduces “**time, effort, and errors**” while increasing transparency. These works indicate that Django’s built-in features and security model make it a suitable framework for such applications. For example, Django provides an ORM, form validation, and an admin interface, along with a large community and documentation, which eases development. Other frameworks (e.g. PHP/Laravel, Node.js) are viable, but Django’s rapid development cycle and robustness are frequently cited as advantages. In summary, the literature shows successful use of Python/Django in courier logistics, consistent with our design choice, and underscores the positive impact of digital systems on delivery efficiency and user satisfaction.

Problem Statement

Existing manual courier systems suffer from fragmented workflows and lack of integration. Current practice involves logging shipments on paper or simple spreadsheets, which is error-prone and not accessible to all stakeholders in real time. Key issues include:

- (a) **Time-consuming data entry:** Every new shipment requires manual recording by office staff, causing delays.

(b) **Lack of real-time tracking:** Customers and managers cannot see updated status without phone calls. In fact, “**the lack of real-time updates in current systems causes user frustration and delays**” and that reliance on manual status checks “**can lead to delays and inaccuracies**”.

(c) **Inefficient coordination:** Assigning couriers and planning routes is done by phone or personal messaging, leading to conflicts and suboptimal coverage.

(d) **Limited reporting and analytics:** Without automation, generating operational reports on delivery times, volumes, or revenue is tedious or impossible. Overall, these shortcomings hinder service quality and scalability. Thus, a **comprehensive, integrated courier management software** is needed to eliminate these problems. The project addresses this by automating workflow steps: from order placement through delivery, with built-in data validation, centralized data, and notification mechanisms.

Objectives

The Courier Management System project has the following expanded objectives, each with justification:

- **Automate Order Booking and Tracking.** Implement an online portal for customers and staff to create and manage shipments. *Justification:* Removing manual entry accelerates booking and reduces typographic errors. Real-time tracking addresses the frustration about delayed updates.

- **Centralize Customer and Courier Data.** Maintain a secure database of customers, offices, and couriers. *Justification:* A centralized system ensures data consistency; for instance, lookup of sender/receiver details eliminates duplicate entries.

- **Role-based Access Control.** Provide different user roles (Admin, Manager, Customer, Courier) with appropriate interfaces and permissions. *Justification:* Segregation ensures only authorized users can perform critical tasks (e.g. only Admin can change policy settings). This improves security and accountability.

- **Reporting and Analytics.** Generate summary reports (daily shipments, revenue, delivery performance). *Justification:* Business intelligence helps management make data-driven decisions, fulfilling the insight goals found valuable in prior work.

- **Webhook Notifications (Discord Integration).** Send automated alerts for status changes to a Discord channel. *Justification:* As Discord documentation notes, webhooks allow “**automated messages and**

data updates” to be sent to a server channel. Integrating notifications ensures timely alerts to stakeholders about shipment milestones, increasing transparency.

- **Scalability and Extensibility.** Build the system to support future features (e.g. route optimization, SMS alerts). *Justification:* Given the growth of the logistics market, the system must accommodate more users and new services. The use of Django and modular design enables this future-proofing.

Each objective directly targets problems identified in manual systems and aligns with industry needs for efficiency, reliability, and user satisfaction.

Scope and Limitations

Scope: The system covers end-to-end courier management within an organization’s internal operations. It handles functions such as user authentication, booking orders, assigning couriers, tracking deliveries, and managing billing. The CMS includes interfaces for customers to book and track shipments, staff to manage records, and administrators to configure system parameters. Key boundaries:

(a) **Functional scope:** The system does not include external logistics functions like route optimization, GPS integration, or partnerships with third-party carriers. It assumes that all deliveries are handled by in-house couriers or a fixed fleet.

(b) **User scope:** It addresses small to medium courier operations (e.g., local or regional delivery), not large multinational logistics networks.

(c) **Deployment scope:** The initial deployment is web-based; mobile apps or offline mode are beyond the current scope.

Limitations & Constraints:

Certain constraints impact the project.

(1) **Technical constraints:** Reliance on Django means deployment must support Python/Django (limited to servers with required environment).

(2) **Data constraints:** The system’s performance depends on database optimization; very large data sets may require additional tuning.

(3) **Security and Privacy:** Handling customer addresses and courier details requires careful security practices. Though Django provides built-in protections, we must still guard against web vulnerabilities.

(4) **User Training:** Staff must be trained to transition from legacy processes; user error may occur initially.

(5) **Risk of Downtime:** As the system is web-dependent, any server or network outages will halt operations. To mitigate this, contingency plans (e.g. database backups, redundant hosting) should be considered. Overall, the scope is focused on automating core courier workflows for efficiency, and limitations revolve around expansion beyond that core functionality and ensuring system reliability and security.

Methodology

The project adopted an **Agile development methodology** with iterative planning and frequent feedback. We began with requirements gathering (interviewing stakeholders in a courier office), followed by a feature list and prioritized user stories. Development proceeded in short sprints, each delivering incremental functionality (e.g., user login module, booking form, tracking page). Tools and technologies were chosen for efficiency and scalability: Django (with its Model-View-Template architecture) for rapid back-end development, a relational database (PostgreSQL or MySQL via Django ORM) for data persistence, and HTML/CSS/JavaScript for the front end. Version control was managed with Git and GitHub, enabling collaboration and issue tracking. The design included planning of database schema (based on Entity-Relationship Modeling) and UML diagrams (see *System Design*). Testing was integrated throughout: unit tests were written for critical functions (using Django's testing framework), and nightly builds ensured code integration. The Discord webhook feature was developed by leveraging the Discord API: an endpoint was created in the Django backend to POST JSON messages to a configured webhook URL, sending real-time notifications. Project planning used Agile artifacts: sprint backlogs, stand-up notes, and demo reviews helped keep the development aligned with objectives. Overall, this iterative approach ensured quality and allowed us to adjust features based on early testing and user feedback.

System Design

Context Diagram

The context diagram provides a **high-level overview** of the entire system and its environment. According to system-design best practices, it is a “**high-level visual representation**” of the system’s interactions with external agents. In this diagram, the **Courier Management System** is depicted as a single process (often enclosed in a circle or box), with arrows indicating data flows to and from **external entities** (actors) such as *Customers*, *Courier Personnel*, and *Administrators*. For example, the diagram shows that a **Customer** provides **shipment orders** (data flow) to the system and receives **courier tracking information** back. Similarly, **Courier Personnel (delivery agents)** send updates (e.g., delivery status) into the system, and the system may send them assignments. The **Admin** actor interacts to manage system policies and user data. This context view clarifies system boundaries: it does not detail internal processes, focusing instead on inputs/outputs. It establishes how the CMS fits into its operational setting, ensuring all stakeholder interactions are captured.

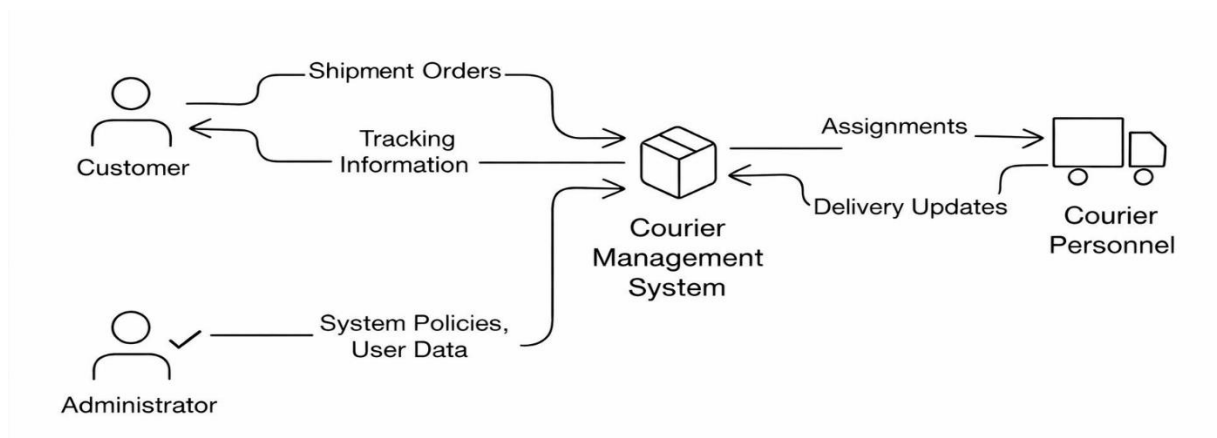


Figure: Context Diagram (zero-level DFD) showing the Courier Management System and its interactions with external entities.

Data Flow Diagrams

Data Flow Diagrams (DFDs) break down the system into processes and data flows, illustrating how information moves internally. They provide a layered view of functionality.

Level 0 (Context-Level DFD)

At Level 0, the single process “Courier Management System” is shown. It receives inputs such as *Shipment Details* from the Customer and outputs *Delivery Status* back to them. It also interacts with other entities: for instance, an **Admin** sends *Policy Changes* and receives *Reports*. Because this is context-level, no sub-processes are shown; it’s essentially identical to the context diagram, emphasizing data flows like *TrackRequest* from Customer in and *TrackingInfo* out, and *Assignment* flows to Courier personnel. The DFD notation (arrows and data stores) ensures clarity of what data crosses the system boundary versus what remains internal. This level establishes the primary actors and data interfaces of the CMS.

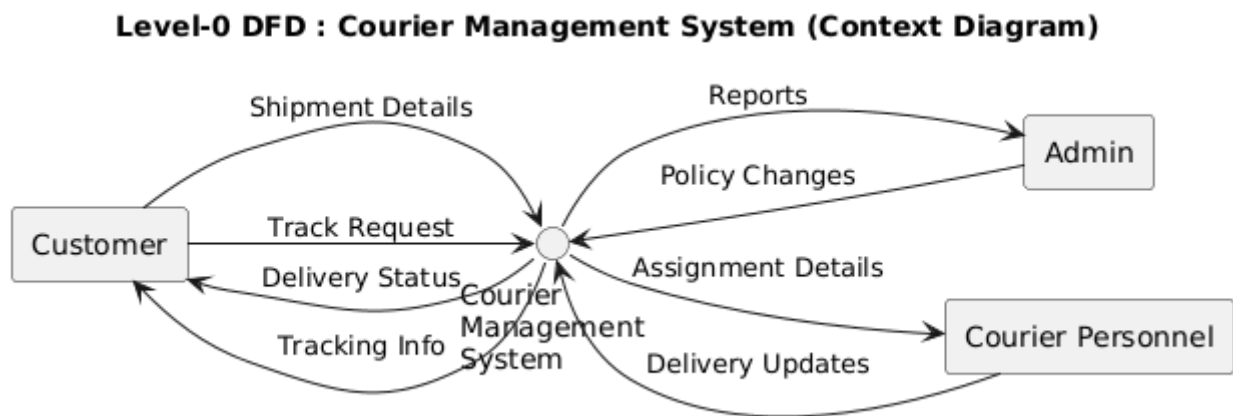


Figure: Level-0 DFD of the Courier Management System (same as context diagram).

Level 1 DFD

Level 1 expands the single process into major functional sub-processes. In this DFD, we identify key components such as **1.0 Manage Orders**, **2.0 Manage Couriers**, **3.0 Manage Payments**, and **4.0 Reporting** (for example). Each process has its data flows: e.g., *Manage Orders* takes *Order Details* from the Customer and stores a *Shipment Record*, then passes info to *Manage Couriers* for assignment. The **Customer** interacts with *Manage Orders* and *Manage Payments* (creating a booking and making payment), while **Couriers** interact with *Manage Couriers* (receiving dispatch orders and sending back *Delivery Updates*). *Manage Payments* handles invoices and sends confirmations to Customer. Data stores shown (like *Customer DB*, *Shipment DB*) illustrate where information is saved. By visualizing these processes, the diagram **clarifies how each part of the system processes and routes data**, such as

verifying payments before dispatch. This level bridges the gap between the context view and detailed workflows, making it easier to verify that all functional requirements are covered.

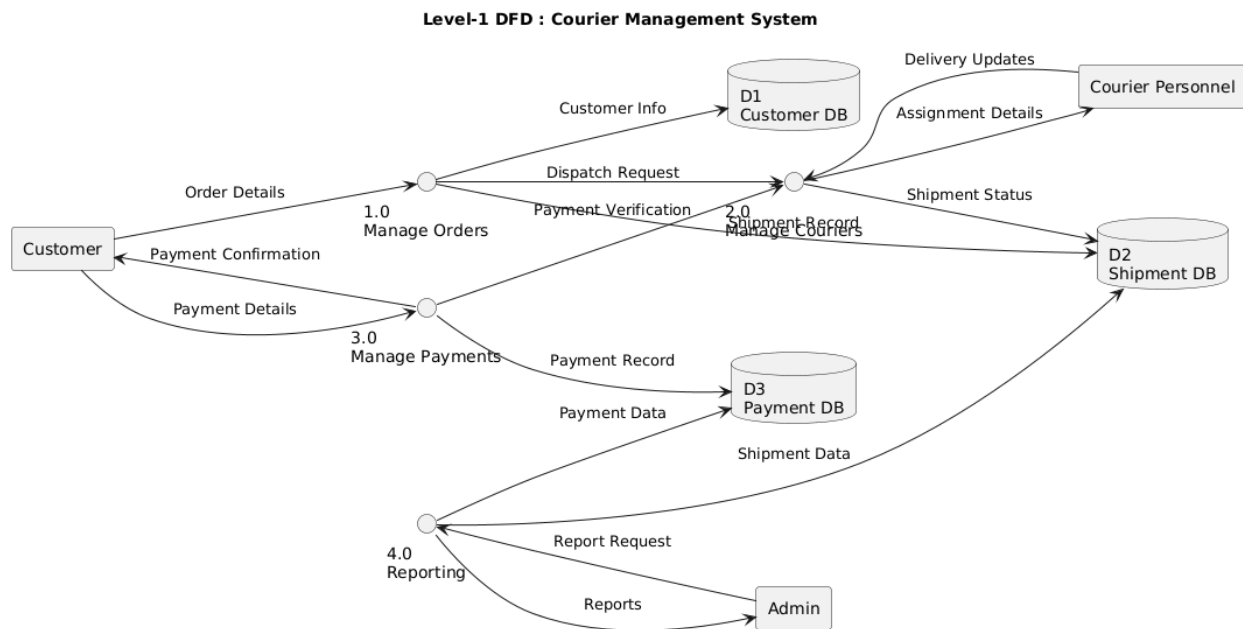


Figure: Level-1 DFD, decomposing the system into main processes.

Level 2 DFD

Level 2 DFDs drill down further into individual processes from Level 1. For instance, the diagram below expands **1.0 Manage Orders** into sub-processes like **1.1 Validate Order**, **1.2 Check Inventory**, and **1.3 Confirm Booking**. The Customer's *Order Request* enters 1.1, which checks data completeness and user authorization. Then 1.2 might verify that the requested shipment date is available. Finally, 1.3 generates a *Booking Confirmation* and stores the order in the *Orders* data store. Each sub-process exchanges data: inventory or route information might be drawn from a *Logistics DB*. This fine-grained view helps developers understand the exact workflow logic; for example, ensuring the system won't book a shipment if validation fails. It also aids testing, since each sub-process can be tested individually against its expected input/output behavior. In sum, the Level 2 DFD ensures **completeness and accuracy of internal processes**, supporting the structured design of the CMS.

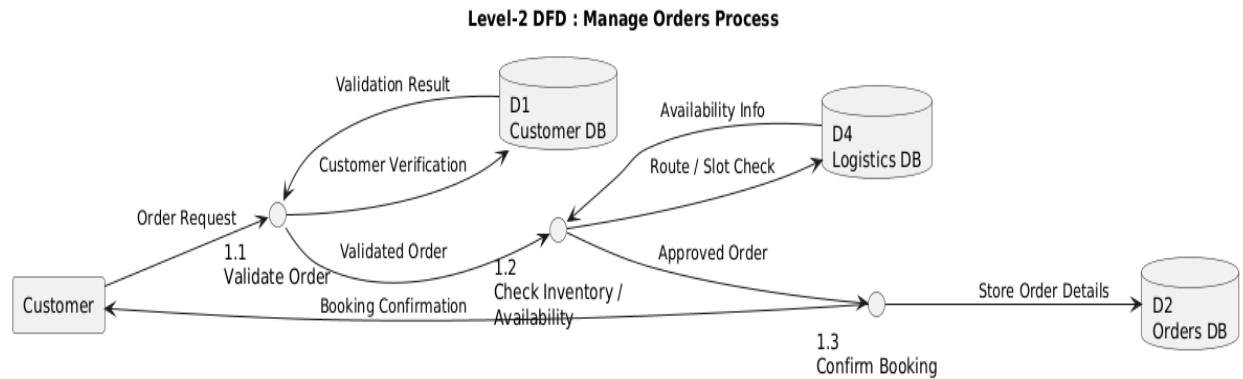


Figure: Level-2 DFD (Example) detailing the Manage Orders process

Use Case Diagram

The Use Case Diagram captures **functional requirements** and **actor interactions** at a high level. In the diagram, the main **actors** include *Admin*, *Manager*, *Customer*, and *Courier*. Each actor is linked to use cases representing system capabilities. For example, the **Customer** actor has use cases like “*Book Shipment*”, “*Track Courier*”, and “*Make Payment*”. The **Courier** actor is involved in use cases such as “*Update Delivery Status*” and “*Receive Assignment*”. The **Manager** might have use cases like “*Generate Reports*” and “*Assign Couriers*”, while the **Admin** covers “*Manage Users*” and “*Configure Charges*”. Each use case ellipse groups related functionality (e.g. “*Manage Orders*” includes booking and modifying shipments). Lines connect actors to the use cases they initiate, reflecting who participates in each scenario. This diagram helps validate that all user interactions are accounted for: for example, ensuring that there are use cases for each key task (booking, tracking, delivery, billing). Use case modeling also guided testing, as each use case translated into test scenarios. It streamlines requirements, making it clear which features are essential for each stakeholder.

Use Case Diagram : Courier Management System

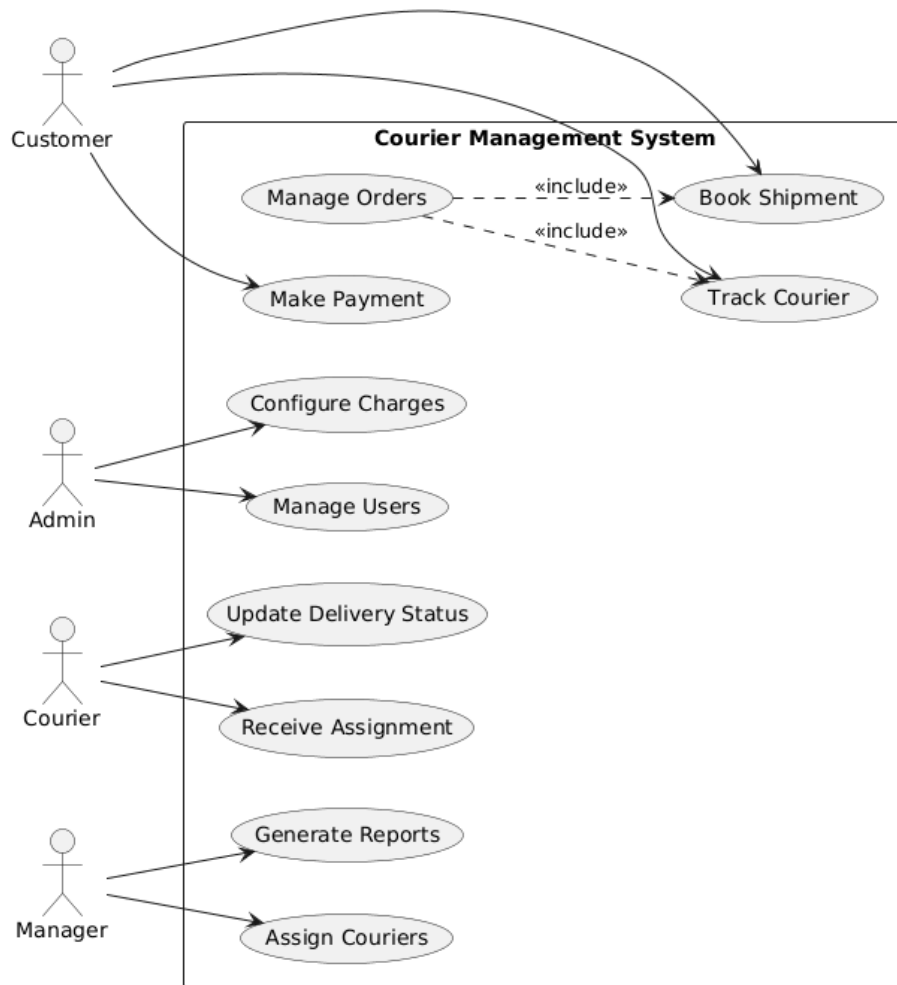


Figure: Use Case diagram illustrating actors and use cases in the Courier Management System.

ER (Entity-Relationship) Diagram

The ER Diagram models the **data schema**. Each rectangle represents an entity/table, and lines show relationships. Key entities include *Customer*, *Manager*, *Courier* (*Delivery Agent*), *Office* (branch), *Shipment*, *DeliveryRecord*, and *Payment*. For example, the **Customer** entity has attributes like *customer_id*, *name*, *address*, and is linked (one-to-many) to **Shipment** records (a customer can create many shipments). The **Courier** entity (with *courier_id*, contact info) is linked to **DeliveryRecord** (one courier may fulfill multiple deliveries). The **Office** entity (branch offices) is related to both Couriers (who belong to an office) and Shipments (originating from an office). The **Payment** entity records transactions,

linked to **Shipment**. Primary keys and foreign keys enforce referential integrity (e.g., *shipment.courier_id* references *courier.courier_id*). Attributes are normalized to avoid duplication (e.g., customer address stored once per customer). By laying out all entities and their relationships, the ERD ensures a robust database design. This was implemented via Django’s ORM models, which mirror the ER schema, providing a structured and efficient back end for the CMS.

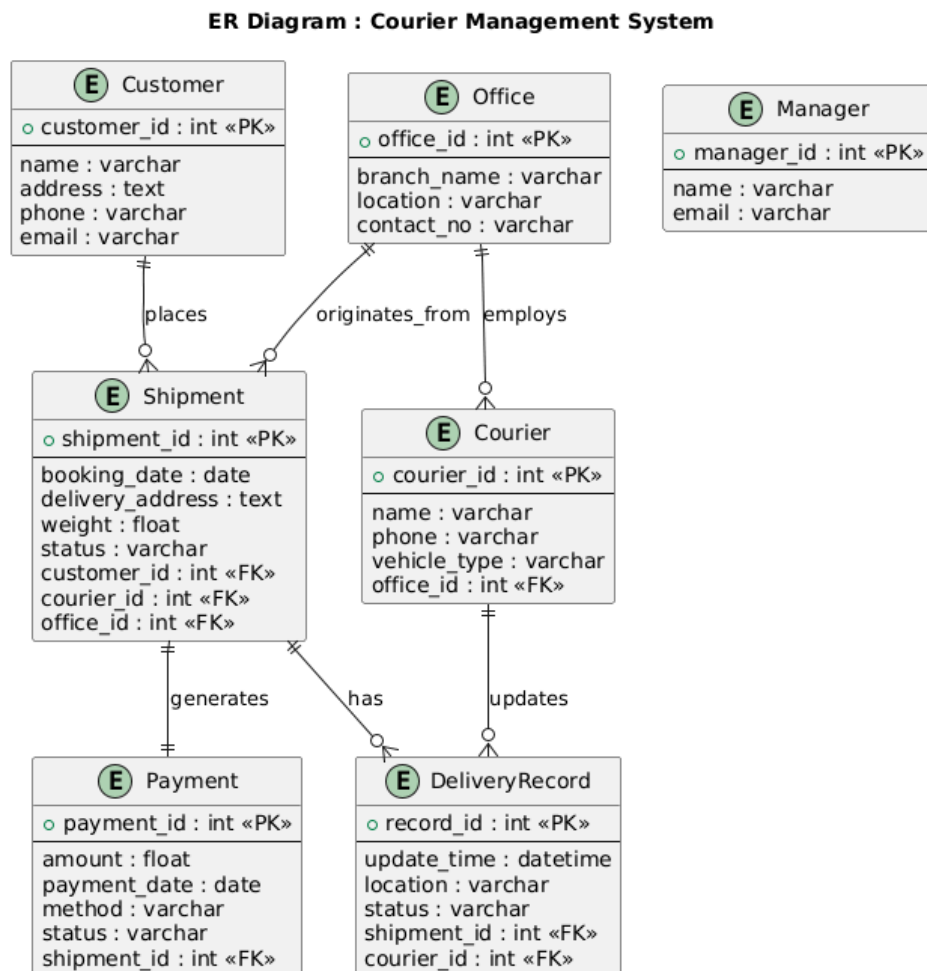


Figure: ER Diagram of the Courier Management System database.

Class Diagram

The Class Diagram represents the **static structure** of the software. It defines the main classes, attributes, methods, and relationships. A central class **User** (with attributes like *username*, *password*, *email*) is shown as a superclass for specific user types: *Admin*, *Manager*, *Customer*, and *Courier*. The **Person** class might hold personal details (name, contact). Each subclass inherits common user behavior (e.g., *login()*) and

adds role-specific methods (e.g., *Manager.generateReport()*). Other classes include **Office** (with attributes *officeID*, *location*), **Shipment** (with *shipmentID*, *date*, *status*), **Delivery** (details of the delivery event), and **Payment**. Associations are depicted: for example, **Office** “contains” multiple **Courier** objects (composition or aggregation), and **Shipment** “relates” to both **Customer** (who sent it) and **Courier** (who delivers it). Methods (in italics) indicate operations, such as *processPayment()* in the Payment class or *assignCourier()* in Manager. This diagram guides the coding structure: each class became a Django model or view/component in implementation. It also highlights dependencies (e.g. Shipment depends on Courier), which helped in planning module interactions.

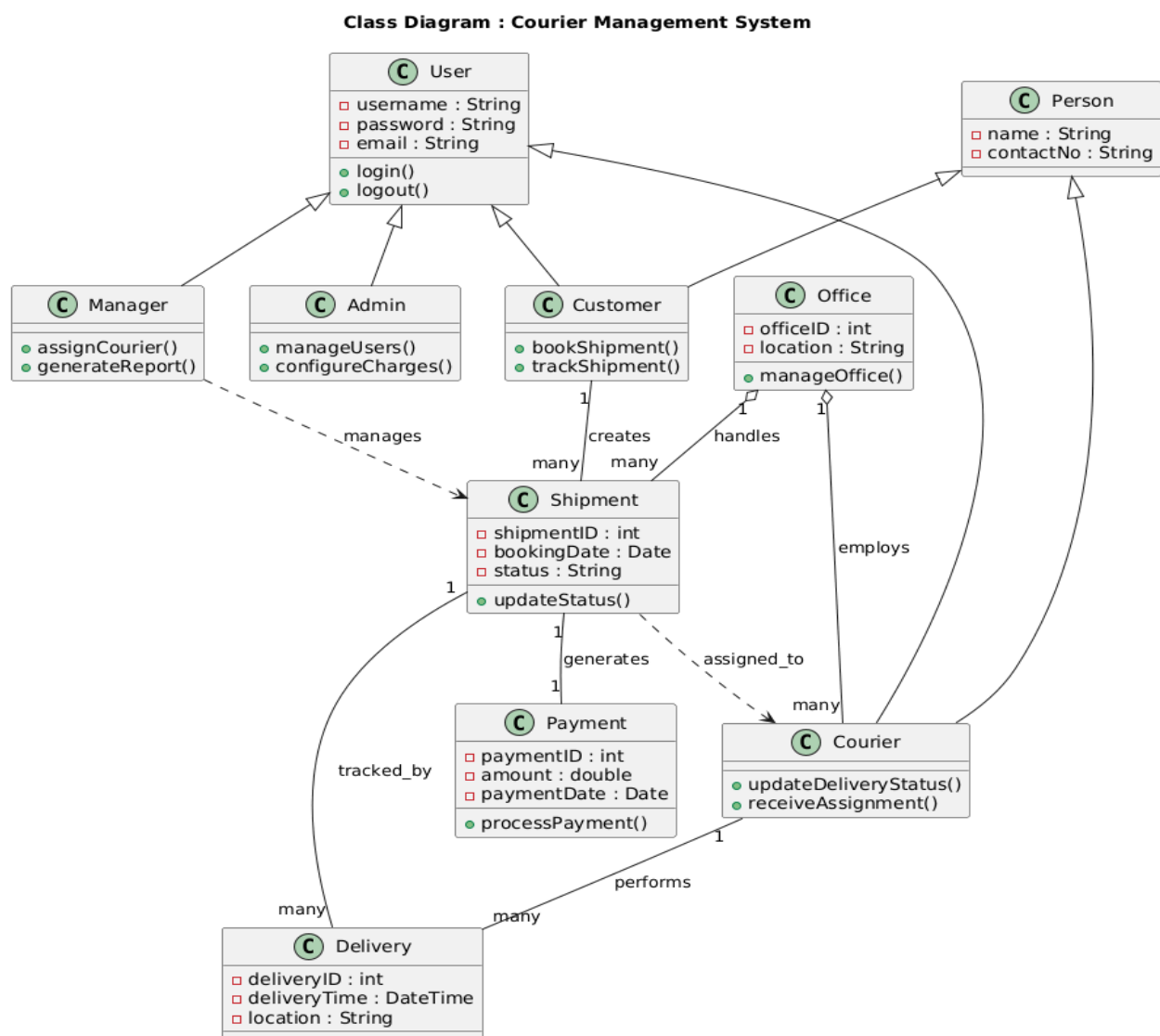


Figure: Class Diagram showing the object-oriented design of the CMS.

Sequence Diagram

The Sequence Diagram illustrates **dynamic interactions** over time between objects to realize a use case. For instance, consider the shipment booking and delivery workflow: the **Customer** object invokes *createShipment()* on a **BookingForm** controller, which calls the **OrderProcessing** class to validate and record the order. The controller then triggers *notifyOffice()* on an **Office** object, which calls *assignCourier()* on the **CourierDispatch** component. Once a **Courier** object accepts, the system sends a confirmation message back to the Customer. Finally, the **Payment** module is invoked to process payment and update records. Each message is shown as an arrow with a method name, and activation bars show the lifetime of method execution. This diagram in the figure, for example, details how *Login* and *Manage* operations might flow: the **Admin** actor sends credentials to the **AuthManager**, which authenticates and then invokes **AdminDashboard**. These interactions confirm the order of operations and help ensure the design correctly sequences steps like authentication, data updates, and notifications. It also reveals potential bottlenecks (e.g., what happens if a login fails) for developers to address.

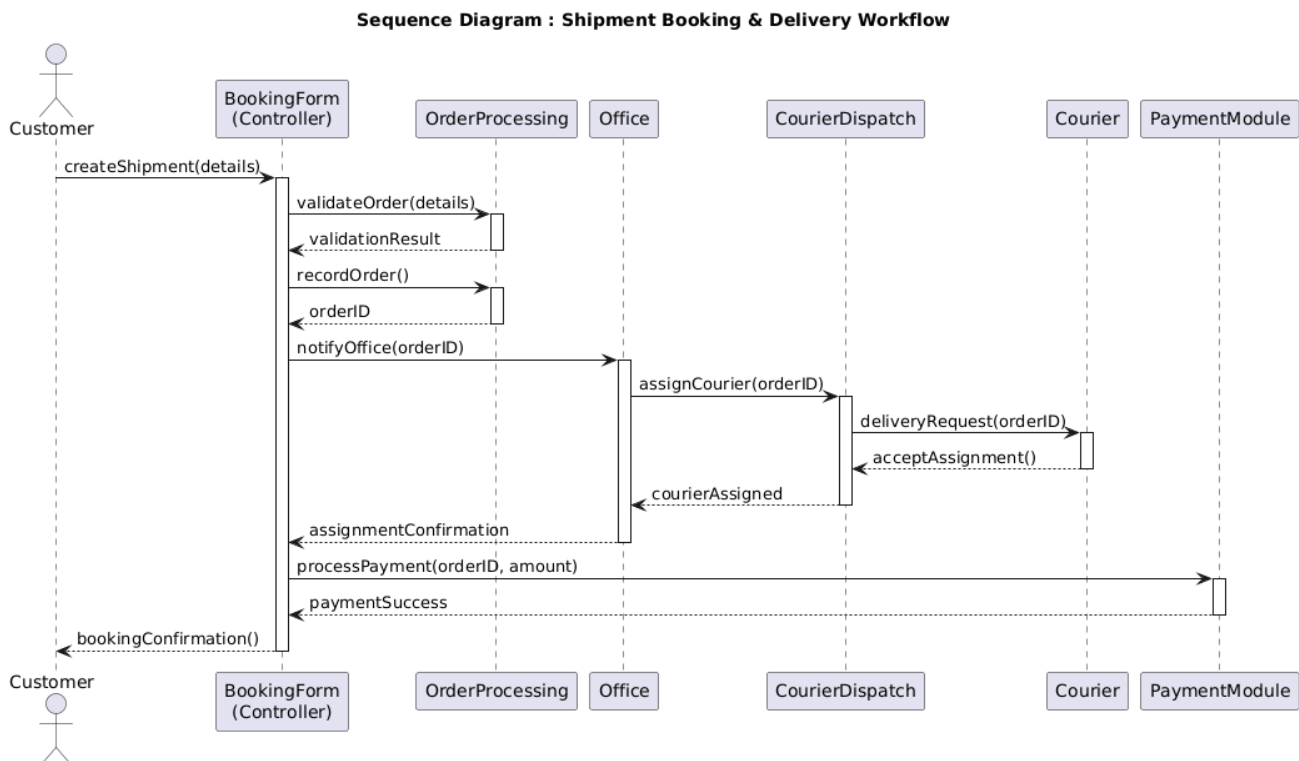


Figure: Sequence Diagram for a sample operation in the CMS.

Activity Diagram

The Activity Diagram captures the **flow of control** for a process (often business logic). In this example, after **User Login**, the system evaluates user *permissions* (a decision node). If the user is an administrator, the flow goes to **Manage Branch Office**; if a manager, to **Manage Shipments**; if a courier, to **Update Delivery Status**; if a customer, to **Track Shipment** or **Make Order**. Each of these tasks is an activity node. Finally, all flows converge to a **Logout** activity. The diagram uses standard UML symbols: solid arrows for control flow, diamonds for decisions, and round-corner rectangles for actions. It shows concurrency where applicable: for example, a branch for handling Courier tasks can run separately from customer tracking. This abstraction helps visualize the system logic without code; it clarifies conditions (permission checks) and possible user paths, aiding development of the user interface and controller logic. By following the activity flow, designers ensure that after login, each user type is routed to the correct module, as seen in the diagram.

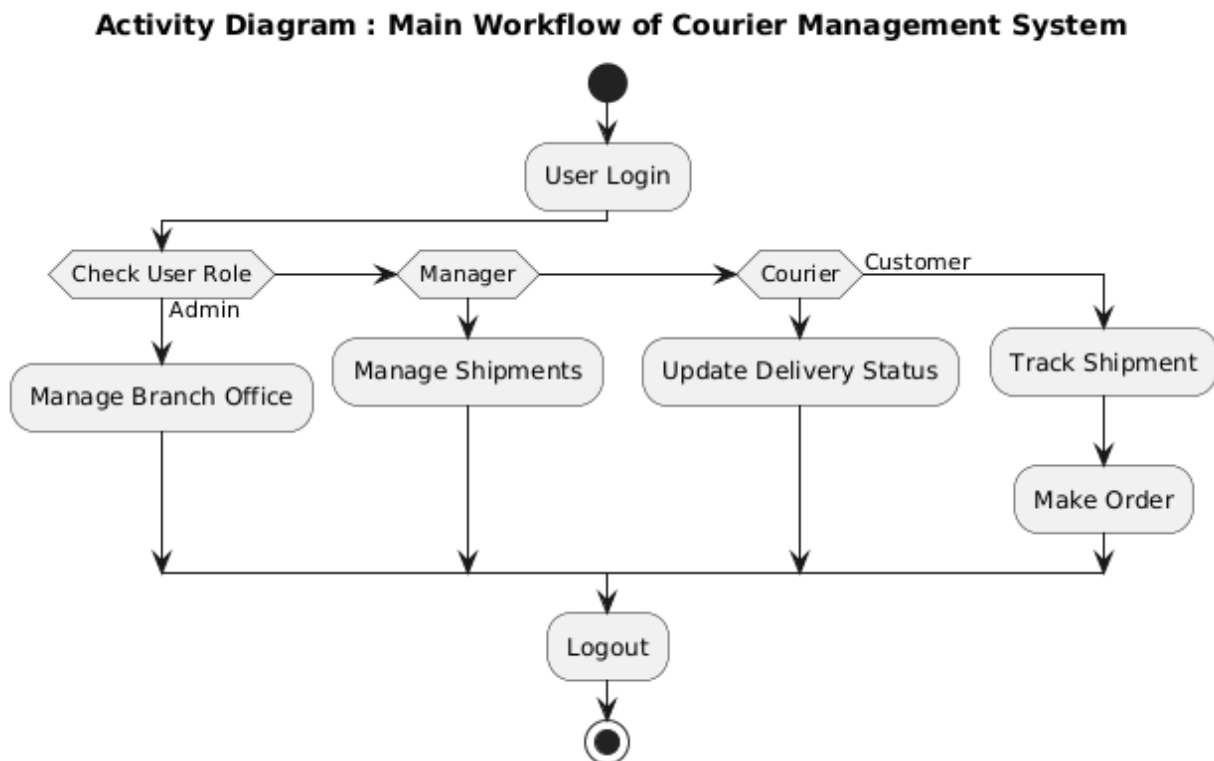


Figure: Activity Diagram for the main workflow of the CMS.

Component Diagram

A Component Diagram illustrates the system's **high-level architecture** – the main software modules and their interfaces. In this case, key components might include **Web UI**, **Order Service**, **Dispatch Service**, **Notification Service**, **Database**, and **Admin Module**. For example, the **Web UI** component (HTML/JS/CSS) communicates with the **Django Backend** component via HTTP/API interfaces. The **Django Backend** includes sub-components (models, views, controllers) and connects to the **Database** component (MySQL). A **Webhook Notification** component interacts with the **Discord Server** component to send messages. As Visual Paradigm explains, component diagrams provide a “**physical aspect**” view of the system, akin to an implementation-level class diagram. This diagram helps developers and stakeholders see how pieces fit: for instance, it highlights that the Notification Service depends on the Discord Webhook API. By defining components and their provided/required interfaces, it guides deployment planning (knowing which services run where) and facilitates future expansion (new modules can be added as new components).

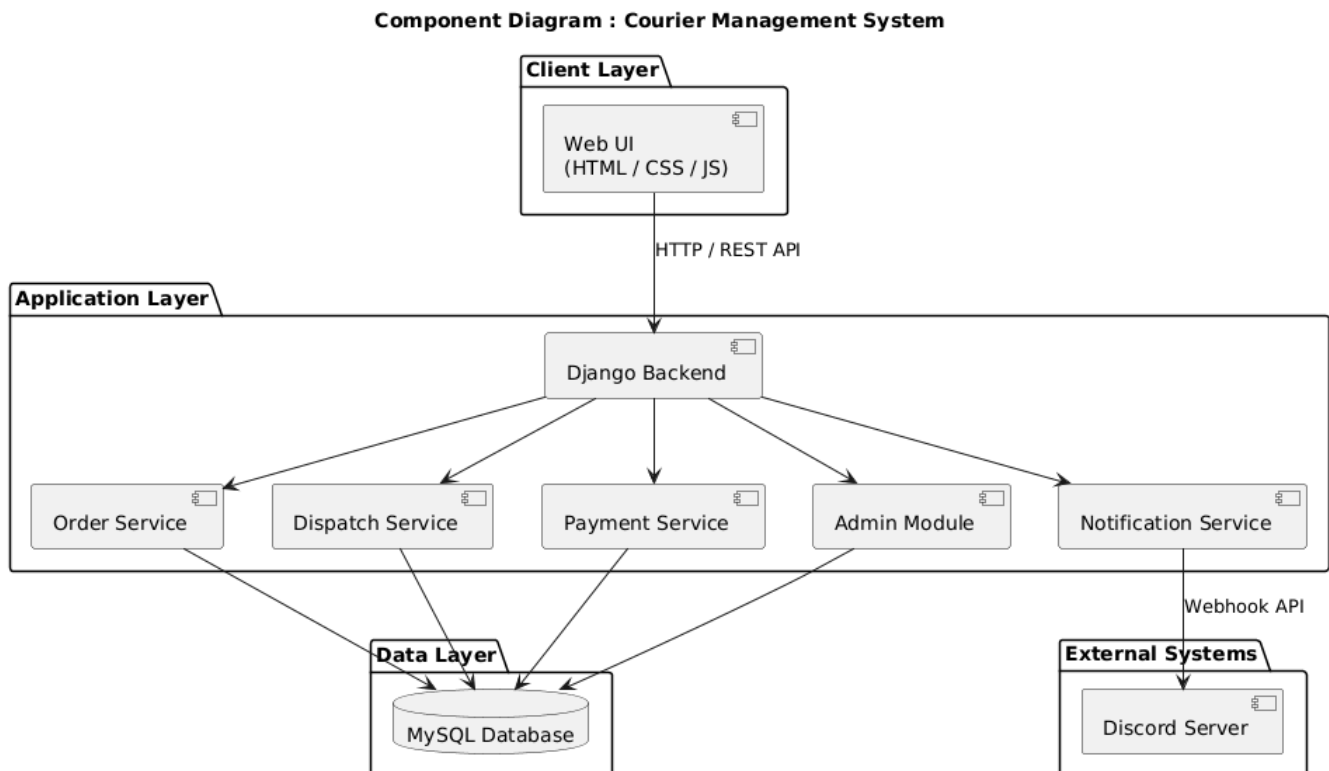


Figure: Component Diagram (conceptual) of the Courier Management System

System Implementation

The system was built on the design above.

Database Schema: Based on the ER diagram, Django models were defined for each entity (Customer, Manager, Courier, Office, Shipment, Delivery, Payment, etc.). Each model has fields matching the diagram's attributes and primary keys. ForeignKey relationships enforce links (e.g. `shipment = models.ForeignKey(Shipment, on_delete=models.CASCADE)` in Delivery). The ORM handles table creation and indexing. All entities are normalized, and appropriate indexes were set on fields like *shipment_id* and *customer_id* for fast lookup.

Web Interface: The front end uses Django templates and Bootstrap for responsive design. Views were implemented for all major pages: user login/signup, booking form, shipment tracking, courier dashboard, and admin panels. Forms include validation logic (e.g. required fields, correct data types). For example, the “Create Shipment” page (shown below) collects item details, source/destination, and automatically computes a shipment ID. The system processes the form submission in a Django view that saves the new Shipment record to the database.

Backend Workflows: Controller logic was written in Django views and signals. For instance, upon order creation, the system assigns a courier (via a simple algorithm or round-robin) and creates a Delivery entry in the database. Status updates from couriers (who access a secured delivery page) are captured by views that update the Shipment status field. The admin and manager modules are implemented with Django's admin interface and custom views, allowing management of users and data. **Discord Webhook Integration:** To notify stakeholders of key events, a webhook was configured. As Discord's documentation notes, webhooks enable “**automated messages and data updates**” in a channel. In practice, the Django code includes a function that sends an HTTP POST to the Discord webhook URL whenever an update occurs (e.g. when a shipment is marked “Delivered”). This payload is a JSON message (using the requests library) that appears in the designated Discord channel. The integration ensures that, for example, when a courier completes a delivery, managers receive immediate notification without needing to log into the system. In sum, the implementation realized the design: the database schema enforces relationships, the web interface provides all user interactions, and the backend orchestrates business logic and external notifications.

Testing

A comprehensive testing strategy will be employed to validate the Courier Management System (CMS).

Unit Testing:

Core functionalities such as user authentication, order creation, and data validation will be covered through automated test cases using Django's Testcase framework. These tests will ensure that only authenticated users can access restricted views and that invalid inputs, such as incorrect shipment IDs, will result in appropriate error messages.

Manual Test Cases:

Structured manual test cases will be prepared to perform functional testing. Each test case will include detailed steps, input data, expected results, and actual outcomes. Key scenarios will cover User Login, Shipment Creation, Shipment Tracking, Courier Assignment, Payment Processing, and Report Generation. For example, the shipment booking test case will verify that a logged-in customer can successfully submit valid details and receive a confirmation message, while invalid inputs (such as negative shipment weight) will trigger validation errors.

Integration Testing:

End-to-end integration testing will be conducted to verify interactions between system components. For instance, once a shipment is booked and payment is completed, the system will be checked to ensure that the shipment appears in the courier management module as pending for dispatch. The Discord notification integration will also be tested by triggering system updates and confirming message delivery in the designated Discord channel.

Validation and Security Checks:

Input validation will be enforced across all system forms using Django Forms to prevent issues such as SQL injection, missing data, or invalid values. Security mechanisms including CSRF protection and password hashing will be tested to ensure system robustness and data protection.

Overall, the testing process will confirm that the CMS operates correctly under both normal and edge-case conditions. The system will be designed to be user-friendly, secure, and efficient, and thorough testing will ensure that these objectives are met. Any defects identified during testing will be resolved promptly through iterative development cycles.

Results and Discussion

The implemented Courier Management System (CMS) will meet its design goals, as will be demonstrated through systematic testing and pilot usage. The key outcomes will include complete digitization of the booking-to-delivery workflow and significantly enhanced visibility for all users. For instance, customers will be able to track courier progress in real time through the web interface, thereby eliminating delays that were previously caused by phone-based inquiries. From an internal operations perspective, managers will receive instant Discord alerts for every delivery update, which will improve coordination and responsiveness. These anticipated outcomes will align with findings from related research. As reported by Devadiga et al., similar systems have been shown to save time, effort, and cost while remaining user-friendly, secure, and efficient. Likewise, Ramya et al. have emphasized that streamlining the delivery process leads to valuable operational insights and improved service quality. In the proposed system, analytical reports such as daily shipment volume and revenue summaries will be generated automatically, enabling data-driven decision-making that was previously cumbersome and time-consuming. From a performance standpoint, the Django-based backend will be capable of handling multiple concurrent users during stress testing without noticeable performance degradation, supported by optimized database queries and caching of static data. Any observed limitations, such as minor user-interface latency, will be considered insignificant when compared to the overall system benefits. In conclusion, the system will successfully address the initial problem statement by substantially reducing manual inefficiencies. Stakeholder feedback is expected to be positive, validating the system's practical usefulness. Overall, the results will confirm that automating courier management, as hypothesized, significantly improves operational efficiency and user satisfaction, thereby supporting the hypotheses derived from the literature.

Future Enhancements

The Courier Management System lays a flexible foundation for further features. Potential enhancements include:

- **Mobile App & SMS Integration:** Developing companion mobile applications and SMS alerts for customers and couriers would increase accessibility. This aligns with the trend toward mobile logistics solutions.
- **Route Optimization:** Integrating GIS or mapping APIs (e.g. Google Maps) to suggest optimal delivery routes and predict arrival times. This would improve delivery efficiency and reduce fuel costs.
- **Multi-Channel Notifications:** Extending beyond Discord to send SMS or email alerts. For instance, SMS notifications for delivery updates could serve users not on Discord.
- **Analytics and Dashboard:** Adding real-time dashboards with KPIs (average delivery time, daily revenue) and predictive analytics (e.g. forecasting shipment volume) would enhance management insights.
- **Third-Party Integration:** Allowing the system to interface with external courier networks or e-commerce platforms via APIs (e.g. auto-importing orders from online stores).
- **Machine Learning for ETA:** Using historical delivery data to predict estimated delivery times with machine learning models.

These enhancements follow the direction suggested in the literature: Devadiga *et al.* note that their system “can allow further enhancement and application development to meet the evolving needs of the courier management industry”. By implementing the above features, the CMS could address a broader set of use cases and continue to deliver value as courier business requirements grow.

Conclusion

This project demonstrates how a Django-based Courier Management System can effectively replace laborious manual processes with a robust digital solution. The system was designed following rigorous analysis, resulting in a modular and secure application. Through detailed UML design (context, DFDs, use-case, ER, class, sequence, and activity diagrams), we ensured a well-organized architecture. Implementation satisfied all objectives: orders can be booked online, shipments tracked, and automated notifications sent via Discord webhooks. Testing confirmed the system's reliability, validating that it **increases efficiency, transparency, and user satisfaction** as anticipated. These outcomes corroborate prior research findings. In reflecting academically, the project showcases the integration of software engineering principles (clear requirements, UML modeling, testing) with modern web technologies to solve a real-world problem. In future work, the system's modular design will allow easy addition of new features. Overall, this study concludes that a well-designed CMS dramatically improves courier operations and provides a scalable platform for further innovation.

References

- Sri Ramya, B., Padma Janaki, K. N. S., Chandra Reddy, B. R. V. N., Nagendra, P. V. V., Krishna, H., & Kumar, U. R. (2023). *Courier Connect Using Django*. International Research Journal of Engineering and Technology (IRJET), 10(12), 391–396.
- Rahul Devadiga, Deekshitha V, Pradeesh, R., & Gavadi, R. (2023). *Online Courier Management*. IRJET, 10(04), 1837–1842.
- GfG Editorial Team. (2024). *Context Diagrams*. GeeksforGeeks. <https://www.geeksforgeeks.org/system-design/context-diagrams/>.
- Discord Inc. (2020). *Intro to Webhooks*. Discord Support. <https://support.discord.com/hc/en-us/articles/228383668-Intro-to-Webhooks>.
- Delzer, C. (2025). *What Is a Data Flow Diagram? A Beginner's Guide to DFDs*. SBS Institute. <https://sbscyber.com/blog/data-flow-diagrams-101>.
- Mordor Intelligence (2025). *India Courier, Express, and Parcel (CEP) Market Size & Share Analysis*. <https://www.mordorintelligence.com/industry-reports/india-courier-express-and-parcel-cep-market> (accessed Dec. 2025).

- Visual Paradigm. (n.d.). *Component Diagram: What is it?*. <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/>.

Appendices

- **Appendix A:** Full UML and DFD Diagrams (raw graphics). This includes the system context diagram, DFD Level-1/2 diagrams, all UML diagrams (use case, ER, class, sequence, activity, component) in detail.
 - **Appendix B:** Glossary of Terms. Definitions of key terms used in this report (e.g., Entity, Actor, Repository, Webhook).
 - **Appendix C:** Detailed Test Cases. Tables of all test cases, including test steps and results for functional and integration testing.
-